Regularization

Ridge and Lasso

Dr. Jomana Bashatah
Slides adapted from Carsten Lange

What Will You Learn/Review

- The **basic idea behind regularization**
- The difference between the **penalty terms for Lasso and Ridge** regression models
- How the **target function for Lasso** regularized regression models differs from the MSE function of an unregularized model
- How to create a **workflow for a Lasso** regularized regression using the *R* `tidymodels` framework
- How the **target function for Ridge** regularized regression model differs from the MSE function of an unregularized model
- How to create a **workflow for a Ridge** regularized model using the *R* `tidymodels` framework

Packages Needed (Already Installed in the background)

- tidyverse
- tidymodels
- glmnet

Loading the Libraries, Data, and Splitting in Training/Testing Data:

```
library(tidymodels); library(rio); library(janitor)
DataHousing = import("https://raw.githubusercontent.com/jomanab-cloud/ISE351-25-Presentations/80f7609c6bcb4887517dcaff62908459cee33e33/HousingData.csv") |>
  clean_names("upper_camel") |>
  select(Price, Sqft=SqftLiving)

set.seed(777)
Split001=initial_split(DataHousing, prop=0.001, strata=Price)
DataTrain=training(Split001)
DataTest=testing(Split001)
print(Split001)
<Training/Testing/Total>
<20/21593/21613>
```

Later, we will use `regularization` to **mitigate the overfitting problem**

The Model

$$\widehat{Price} = \beta_1 Sqft + \beta_2 Sqft_i^2 + \beta_3 Sqft_i^3 + \beta_4 Sqft_i^4 + \beta_5 Sqft_i^5 + \beta_6$$

Benchmark: Unregularized Model

Minimizes the MSE by Choosing the Optimal βs

$$MSE = \frac{1}{20}\sum_{i=1}^{20}\left(\widehat{Price}_i - Price_i\right)^2$$

with:

$$\widehat{Price} = \beta_1 Sqft + \beta_2 Sqft_i^2 + \beta_3 Sqft_i^3 + \beta_4 Sqft_i^4 + \beta_5 Sqft_i^5 + \beta_6$$

Running the Unregularized Benchmark Model

```
RecipeHouses=recipe(Price~., data=DataTrain) |>
step_mutate(Sqft2=Sqft^2,Sqft3=Sqft^3,Sqft4=Sqft^4,Sqft5=Sqft^5) |>
  step_normalize(all_predictors())

ModelDesignBenchmark=linear_reg() |>
  set_engine("lm") |>
  set_mode("regression")

WFModelBenchmark=workflow() |>
  add_model(ModelDesignBenchmark) |>
  add_recipe(RecipeHouses) |>
  fit(DataTrain)

tidy(WFModelBenchmark) #prints the betas in a tidy format
# A tibble: 6 × 5
  term          estimate  std.error statistic       p.value
  <chr>            <dbl>      <dbl>     <dbl>         <dbl>
1 (Intercept)    534105.     41384.    12.9   0.00000000365
2 Sqft          2518634.  10629789.     0.237 0.816
3 Sqft2       -12684915.  61935625.    -0.205 0.841
4 Sqft3        27231501. 150397216.     0.181 0.859
5 Sqft4       -25638849. 166424593.    -0.154 0.880
6 Sqft5         8851162.  67418554.     0.131 0.897
```

Benchmark: Assessing Prediction Quality

Training Data

```
DataTrainWithPredBenchmark=augment(WFModelBenchmark, DataTrain)
metrics(DataTrainWithPredBenchmark, truth=Price, estimate=.pred)
# A tibble: 3 × 3
  .metric .estimator  .estimate
  <chr>   <chr>           <dbl>
1 rmse    standard   154844.
2 rsq     standard        0.738
3 mae     standard   135245.
```

Testing Data

```
DataTestWithPredBenchmark=augment(WFModelBenchmark, DataTest)
metrics(DataTestWithPredBenchmark, truth=Price, estimate=.pred)
# A tibble: 3 × 3
  .metric .estimator    .estimate
  <chr>   <chr>             <dbl>
1 rmse    standard   3881459.
2 rsq     standard         0.0243
3 mae     standard    208027.
```

Regularization

Ridge

$$\boldsymbol{T^{arget} = \frac{1}{20}\sum_{i=1}^{20}\left(\widehat{price}_i - price_i\right)^2 + \lambda P^{enalty}}$$

With:

$$\widehat{Price} = \beta_1 Sqft + \beta_2 Sqft_i^2 + \beta_3 Sqft_i^3 + \beta_4 Sqft_i^4 + \beta_5 Sqft_i^5 + \beta_6$$

With:

$$\boldsymbol{P^{enalty} = \sum_{j=1}^{5}\beta_j^2}$$

Two Goals: Minimize MSE and Minimize Penalty (small or zero βs)
T^{arget} value still only depends on data and βs.

Note, when reducing a large and a small parameter by the same amount, the impact of reducing the large parameter has a bigger impact on the *penalty* than reducing the small parameter. Thus. **Ridge has a tendency to reduce large rather than small parameters**

Running the Ridge Model

```
library(glmnet)
set.seed(777)
ModelDesignRidge=linear_reg(penalty=1000000, mixture=0) |> #0 for ridge, 1 for lasso
  set_engine("glmnet") |>
  set_mode("regression")

WFModelRidge=workflow() |>
  add_model(ModelDesignRidge) |>
  add_recipe(RecipeHouses) |>
  fit(DataTrain)

tidy(WFModelRidge)
# A tibble: 6 × 3
  term        estimate penalty
  <chr>          <dbl>   <dbl>
1 (Intercept)  534105  1000000
2 Sqft          34941. 1000000
3 Sqft2         33781. 1000000
4 Sqft3         31484. 1000000
5 Sqft4         29539. 1000000
6 Sqft5         28219. 1000000
```

Comparing the Coefficients (βs)

Benchmark Coefficients (βs)

```
# A tibble: 6 × 5
  term          estimate  std.error statistic       p.value
  <chr>            <dbl>      <dbl>     <dbl>         <dbl>
1 (Intercept)    534105.     41384.    12.9   0.00000000365
2 Sqft          2518634.  10629789.     0.237 0.816
3 Sqft2       -12684915.  61935625.    -0.205 0.841
4 Sqft3        27231501. 150397216.     0.181 0.859
5 Sqft4       -25638849. 166424593.    -0.154 0.880
6 Sqft5         8851162.  67418554.     0.131 0.897
```

Ridge Model Coefficients (βs)

```
# A tibble: 6 × 3
  term        estimate penalty
  <chr>          <dbl>   <dbl>
1 (Intercept)  534105  1000000
2 Sqft          34941. 1000000
3 Sqft2         33781. 1000000
4 Sqft3         31484. 1000000
5 Sqft4         29539. 1000000
6 Sqft5         28219. 1000000
```

Assessing Prediction Quality Benchmark & Ridge Model (Training Data)

Benchmark

```
DataTrainWithPredBenchmark=augment(WFModelBenchmark, DataTrain)
metrics(DataTrainWithPredBenchmark, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   154844.
2 rsq     standard       0.738
3 mae     standard   135245.
```

Ridge

```
DataTrainWithPredRidge=augment(WFModelRidge, DataTrain)
metrics(DataTrainWithPredRidge, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   193416.
2 rsq     standard       0.711
3 mae     standard   167423.
```

Assessing Prediction Quality Benchmark & Ridge Model (Testing Data)

Benchmark

```
DataTestWithPredBenchmark=augment(WFModelBenchmark, DataTest)
metrics(DataTestWithPredBenchmark, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   3881459.
2 rsq     standard     0.0243
3 mae     standard    208027.
```

Ridge

```
DataTestWithPredRidge=augment(WFModelRidge, DataTest)
metrics(DataTestWithPredRidge, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   310717.
2 rsq     standard     0.304
3 mae     standard   191110.
```

Regularization

Lasso

$$\boldsymbol{T^{arget} = \frac{1}{20}\sum_{i=1}^{20}\left(\widehat{price}_i - price_i\right)^2 + \lambda P^{enalty}}$$

With:

$$\widehat{Price} = \beta_1 Sqft + \beta_2 Sqft_i^2 + \beta_3 Sqft_i^3 + \beta_4 Sqft_i^4 + \beta_5 Sqft_i^5 + \beta_6$$

With:

$$\boldsymbol{P^{enalty} = \sum_{j=1}^{5} |\beta_j|}$$

Two Goals: Minimize MSE and Minimize Penalty (small or zero βs)
T^{arget} value still only depends on data and βs.

Note, reducing a large or a small β parameter by the same amount has the same impact on the *penalty*. Thus Lasso would always reduce the parameter where the reduction has the smallest impact on the MSE. **Some parameters might be reduced to zero**

Running the Lasso Model

```
library(glmnet)
set.seed(777)
ModelDesignLasso=linear_reg(penalty=500, mixture=1) |>
  set_engine("glmnet") |>
  set_mode("regression")

WFModelLasso=workflow() |>
  add_model(ModelDesignLasso) |>
  add_recipe(RecipeHouses) |>
  fit(DataTrain)

tidy(WFModelLasso)
# A tibble: 6 × 3
  term        estimate penalty
  <chr>          <dbl>   <dbl>
1 (Intercept)  534105      500
2 Sqft          62641.     500
3 Sqft2        202825.     500
4 Sqft3             0      500
5 Sqft4             0      500
6 Sqft5             0      500
```

Comparing the Coefficients (βs)

Benchmark Coefficients (βs)

```
# A tibble: 6 × 5
  term          estimate  std.error statistic       p.value
  <chr>            <dbl>      <dbl>     <dbl>         <dbl>
1 (Intercept)    534105.     41384.    12.9   0.00000000365
2 Sqft          2518634.  10629789.     0.237 0.816
3 Sqft2       -12684915.  61935625.    -0.205 0.841
4 Sqft3        27231501. 150397216.     0.181 0.859
5 Sqft4       -25638849. 166424593.    -0.154 0.880
6 Sqft5         8851162.  67418554.     0.131 0.897
```

Lasso Model Coefficients (βs)

```
# A tibble: 6 × 3
  term        estimate penalty
  <chr>          <dbl>   <dbl>
1 (Intercept)  534105      500
2 Sqft          62641.     500
3 Sqft2        202825.     500
4 Sqft3             0      500
5 Sqft4             0      500
6 Sqft5             0      500
```

Comparing Prediction Quality Benchmark & Lasso Model (Training Data)

Benchmark

```
DataTrainWithPredBenchmark=augment(WFModelBenchmark, DataTrain)
metrics(DataTrainWithPredBenchmark, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   154844.
2 rsq     standard       0.738
3 mae     standard   135245.
```

Lasso

```
DataTrainWithPredLasso=augment(WFModelLasso, DataTrain)
metrics(DataTrainWithPredLasso, truth = Price, estimate = .pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   156602.
2 rsq     standard       0.732
3 mae     standard   138956.
```

Comparing Prediction Quality Benchmark & Lasso Model (Testing Data)

Benchmark

```
DataTestWithPredBenchmark=augment(WFModelBenchmark, DataTest)
metrics(DataTestWithPredBenchmark, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   3881459.
2 rsq     standard     0.0243
3 mae     standard    208027.
```

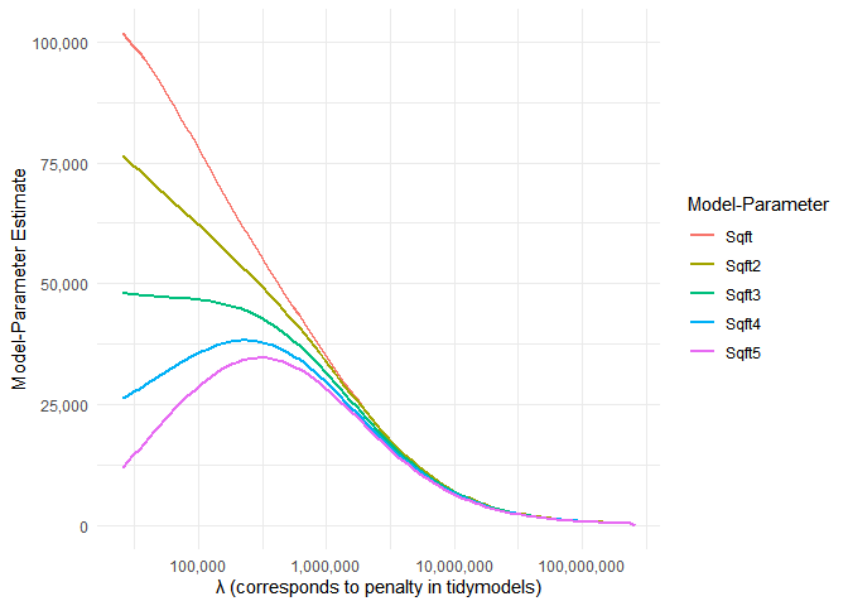
Lasso

```
DataTestWithPredLasso=augment(WFModelLasso, DataTest)
metrics(DataTestWithPredLasso, truth=Price, estimate=.pred)
```

```
# A tibble: 3 × 3
  .metric .estimator .estimate
  <chr>   <chr>          <dbl>
1 rmse    standard   264894.
2 rsq     standard     0.533
3 mae     standard   165158.
```

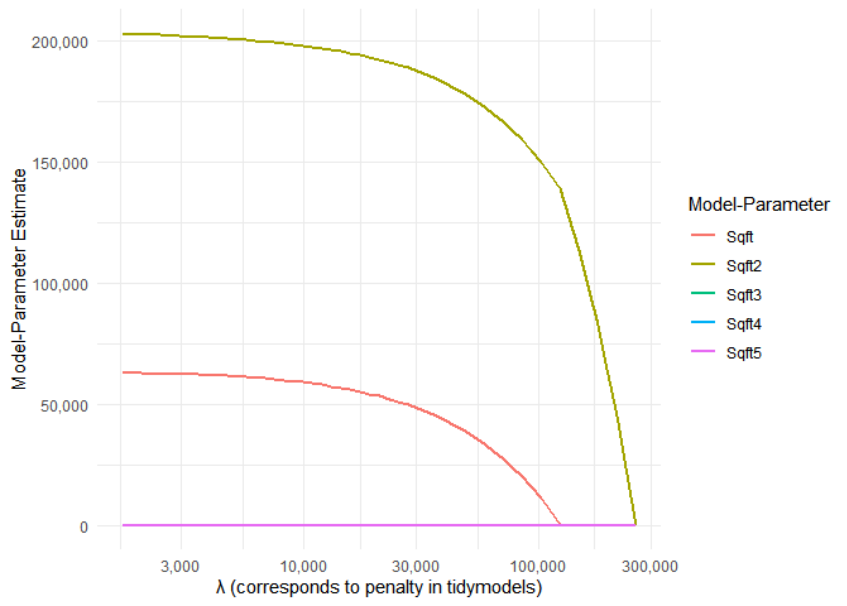
Comparing Parameter Reduction for Different Penalties

Ridge:



Shrinkage of Parameters βs for Increasing Penalty Factors (λ)

Lasso:



Shrinkage of Parameters βs for Increasing Penalty Factors (λ)

Elastic Net

$$T^{arget}_{Elastic} = \underbrace{\frac{1}{20}\sum_{i=1}^{20}\left(\widehat{Price}_i - Price_i\right)^2}_{MSE} + \lambda\overbrace{\left(\alpha P^{enalty}_{Lasso} + (1-\alpha)P^{enalty}_{Ridge}\right)}^{P^{enalty}_{Elastic}}$$

with:

$$\widehat{Price}_i = \beta_1 Sqft_i + \beta_2 Sqft_i^2 + \beta_3 Sqft_i^3 + \beta_4 Sqft_i^4 + \beta_5 Sqft_i^5 + \beta_6$$

$$T^{arget}_{Elastic} = \underbrace{\frac{1}{20}\sum_{i=1}^{20}\left(\widehat{Price}_i - Price_i\right)^2}_{MSE} + \lambda\overbrace{\left(\underbrace{\alpha\sum_{j=1}^{5}|\beta_j|}_{P^{enalty}_{Lasso}} + \underbrace{(1-\alpha)\sum_{j=1}^{5}\beta_j^2}_{P^{enalty}_{Ridge}}\right)}^{P^{enalty}_{Elastic}}$$

Exercise: Tuning Elastic Net

We’ll deliberately use a tiny training sample so overfitting shows up clearly, then fix it with Elastic Net.

Benchmark: Unregularized Model (Overfits on Purpose)

```
set.seed(777)
DataHousing = import("Data/HousingData.csv") |>
  clean_names("upper_camel") |>
  select(Price, Sqft=SqftLiving)
Split = initial_split(DataHousing, prop = 0.02, strata = Price)
DataTrain = training(Split)
DataTest  = testing(Split)

RecipeHouses = recipe(Price ~ Sqft, data = DataTrain) |>
  step_mutate(Sqft2 = Sqft^2, Sqft3 = Sqft^3, Sqft4 = Sqft^4, Sqft5 = Sqft^5) |>
  step_normalize(all_predictors())

ModelBenchmark = linear_reg() |> set_engine("lm") |> set_mode("regression")
WFBenchmark = workflow() |> add_recipe(RecipeHouses) |> add_model(ModelBenchmark) |>
fit(DataTrain)
Train RMSE: 246566
Train Rsq: 0.5794474
Train mae: 176109.4
Test RMSE:  3775667
Test Rsq:  0.02818045
Test mae:  209941
```

Notice the gap between train and test metrics: that gap *is* overfitting.

Specify a Tunable Model

```
ModelTune = linear_reg(penalty = tune(),
mixture = tune()) |>
  set_engine("glmnet") |>
set_mode("regression")
```

`tune()` is a placeholder; we haven't picked penalty or mixture values yet.

Build the Workflow

```
WFTune = workflow() |>
add_recipe(RecipeHouses) |>
add_model(ModelTune)
```

Create Resampling Folds

```
set.seed(777)
Folds = vfold_cv(DataTrain, v = 3)
```

Cross-validation happens entirely inside the training data. The test set stays untouched until the very end.

Define the Candidate Values

```
PenaltyGrid = penalty(range = c(1, 7))
# log10 scale: 10 to 10,000,000
MixtureGrid = mixture(range = c(0, 1))
# 0 = Ridge, 1 = Lasso
```

Build the Full Tuning Grid

```
Grid = grid_regular(PenaltyGrid, MixtureGrid, levels = c(penalty = 7, mixture = 4))
Grid
# A tibble: 28 × 2
    penalty mixture
      <dbl>   <dbl>
 1       10   0
 2      100   0
 3     1000   0
 4    10000   0
 5   100000   0
 6  1000000   0
 7 10000000   0
 8       10   0.333
 9      100   0.333
10     1000   0.333
# ℹ 18 more rows
```

Every row is one penalty/mixture combination we're about to test.

Run the Tuning Search

```
TuneResults = tune_grid(WFTune, resamples
= Folds, grid = Grid)
```

Inspect the Results

```
collect_metrics(TuneResults) |> filter(.metric == "rmse") |> arrange(mean)
# A tibble: 28 × 8
   penalty mixture .metric .estimator    mean     n std_err .config
     <dbl>   <dbl> <chr>   <chr>        <dbl> <int>   <dbl> <chr>
 1  100000   0     rmse    standard   258069.     3  22833. pre0_mod17_post0
 2   10000   1     rmse    standard   260182.     3  23524. pre0_mod16_post0
 3  100000   0.333 rmse    standard   261718.     3  24244. pre0_mod18_post0
 4   10000   0.667 rmse    standard   267890.     3  28241. pre0_mod15_post0
 5  100000   0.667 rmse    standard   271930.     3  30402. pre0_mod19_post0
 6 1000000   0     rmse    standard   276338.     3  20982. pre0_mod21_post0
 7      10   0     rmse    standard   276579.     3  35937. pre0_mod01_post0
 8     100   0     rmse    standard   276579.     3  35937. pre0_mod05_post0
 9    1000   0     rmse    standard   276579.     3  35937. pre0_mod09_post0
10   10000   0     rmse    standard   276579.     3  35937. pre0_mod13_post0
# ℹ 18 more rows
collect_metrics(TuneResults) |> filter(.metric == "rsq") |> arrange(desc(mean))
# A tibble: 28 × 8
    penalty mixture .metric .estimator  mean     n std_err .config
      <dbl>   <dbl> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
 1   100000   1     rsq     standard   0.542     3  0.0898 pre0_mod20_post0
 2   100000   0.333 rsq     standard   0.538     3  0.0923 pre0_mod18_post0
 3   100000   0     rsq     standard   0.538     3  0.0946 pre0_mod17_post0
 4   100000   0.667 rsq     standard   0.538     3  0.0904 pre0_mod19_post0
 5    10000   1     rsq     standard   0.537     3  0.0887 pre0_mod16_post0
 6    10000   0.667 rsq     standard   0.523     3  0.0791 pre0_mod15_post0
 7  1000000   0     rsq     standard   0.517     3  0.0903 pre0_mod21_post0
 8 10000000   0     rsq     standard   0.510     3  0.0901 pre0_mod25_post0
 9       10   0     rsq     standard   0.495     3  0.0592 pre0_mod01_post0
10      100   0     rsq     standard   0.495     3  0.0592 pre0_mod05_post0
# ℹ 18 more rows
```

Select the Best Combination

```
BestParams = select_best(TuneResults,
metric = "rmse")
print(BestParams)
# A tibble: 1 × 3
  penalty mixture .config
    <dbl>   <dbl> <chr>
1  100000       0 pre0_mod17_post0
```

Finalize the Workflow

```
WFFinal = finalize_workflow(WFTune,
BestParams)
```

Refit and Evaluate on Test Data

```
FinalFit = fit(WFFinal, DataTrain)
cat("Tuned Test RMSE:", rmse_vec(DataTest$Price,
predict(FinalFit, DataTest)$.pred), "\n")
```

```
Tuned Test RMSE: 281355.5
```

```
cat("Tuned Test Rsq:", rsq_vec(DataTest$Price,
predict(FinalFit, DataTest)$.pred), "\n")
```

```
Tuned Test Rsq: 0.4452895
```

```
cat("Tuned Test mae:", mae_vec(DataTest$Price,
predict(FinalFit, DataTest)$.pred), "\n")
```

```
Tuned Test mae: 166632
```

Compare this to the benchmark's test RMSE a few slides back. That improvement is what regularization bought you.